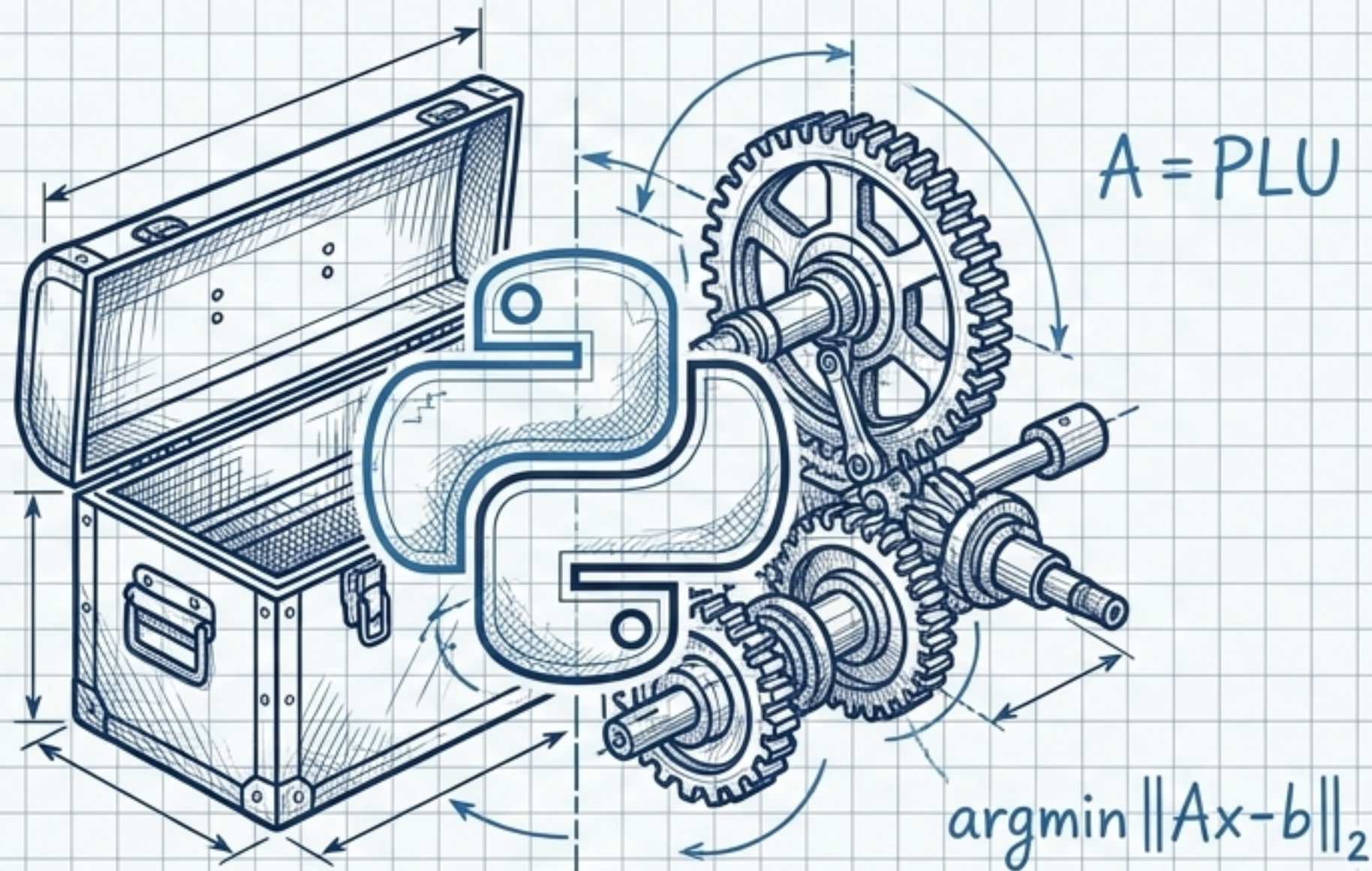


```
import scipy.linalg as la
```

$$Ax=b$$



SciPy 線性代數求解工具箱

從密集矩陣到大型稀疏系統的工程演算法選擇

求解器控制面板 (The Solvers Dashboard)

模塊一：基礎與分析 (Dense/Square)

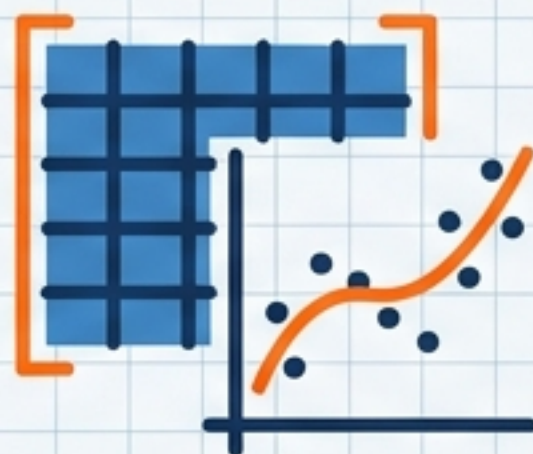


條件：方形矩陣 ($m=n$)、唯一解。

工具：`solve()` (直接求解)

工具：`lu()` (矩陣拆解)

模塊二：妥協與擬合 (Non-Square)



條件：非方形矩陣 (過確定 / 低確定)。

工具：`lstsq()` (最小化殘差)

工具：`pinv()` (最小化操作量)

模塊三：大型與特化 (Sparse)

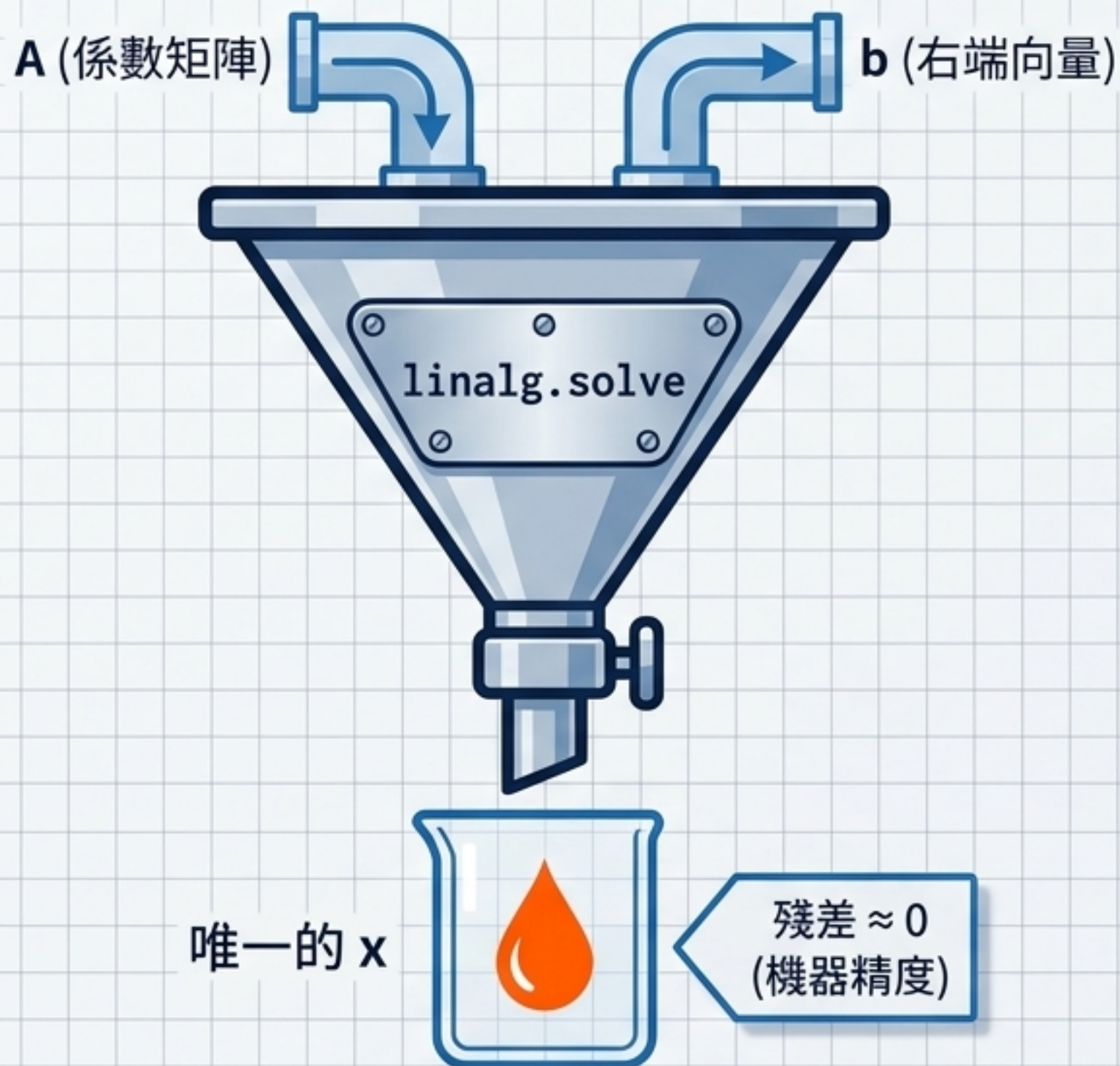


條件：高維度、多數元素為零。

工具：`spsolve()` (直接法)

工具：`cg()`, `gmres()` (迭代法)

基礎作業程序：solve()

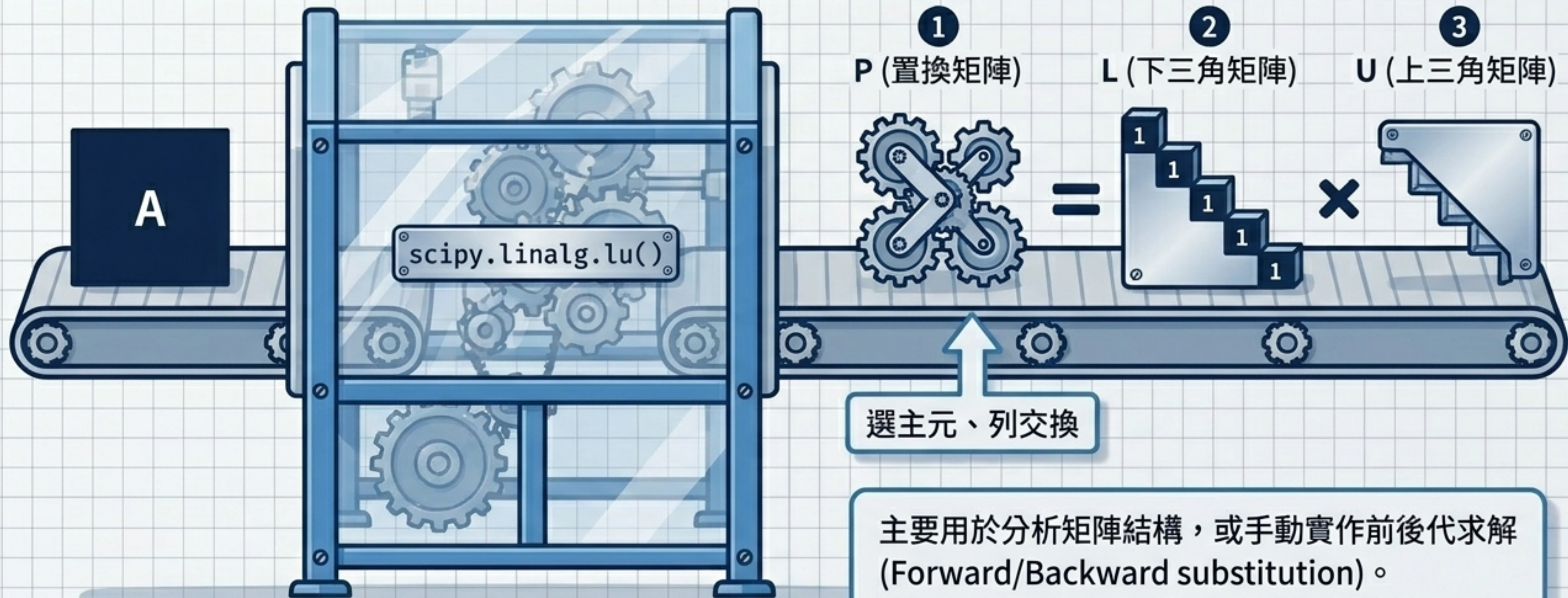


前提條件 (Prerequisites) :

矩陣必須為方形 ($m=n$) 且滿秩 $\det(A) \neq 0$ 。否則將觸發 `LinAlgError`。

```
A = np.array([[2, 1], [1, 3]])  
b = np.array([5, 10])  
x = la.solve(A, b) # Output: [1. 3.]
```


矩陣的解剖學：LU 分解

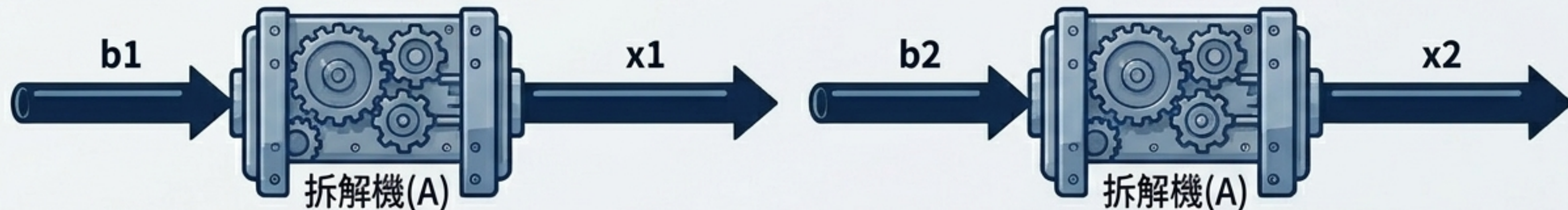


主要用於分析矩陣結構，或手動實作前後代求解 (Forward/Backward substitution)。
注意 SciPy 慣例回傳 P, L, U 滿足 $A = PLU$ 。

效率最大化：重複利用的藝術

每次重造耗時: $O(n^3)$

傳統 solve (Traditional solve)



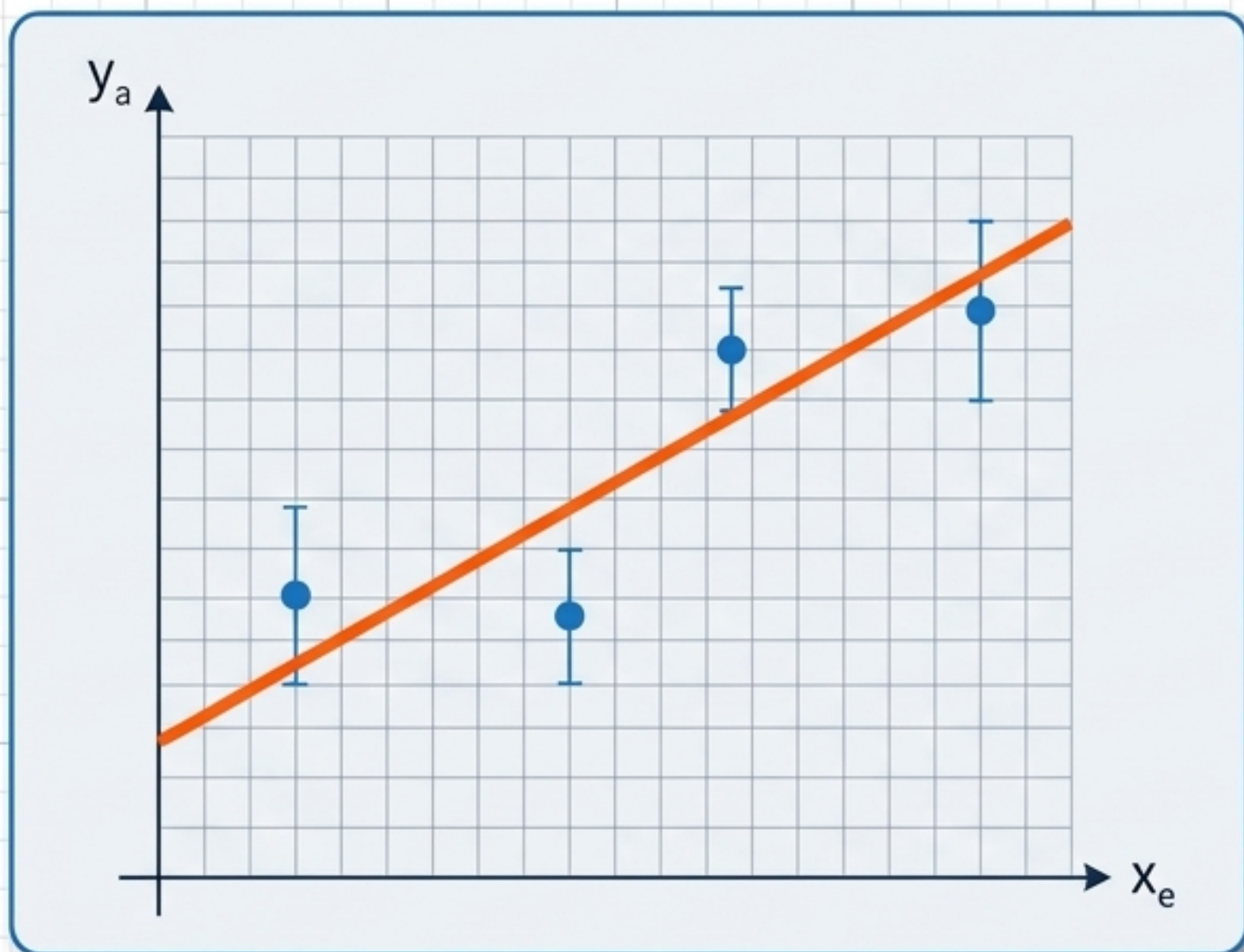
加速比達 $8.66\times$ (測試條件：200x200 矩陣，20 組右端向量。2.72 ms vs 23.53 ms)

lu_factor (Optimized)

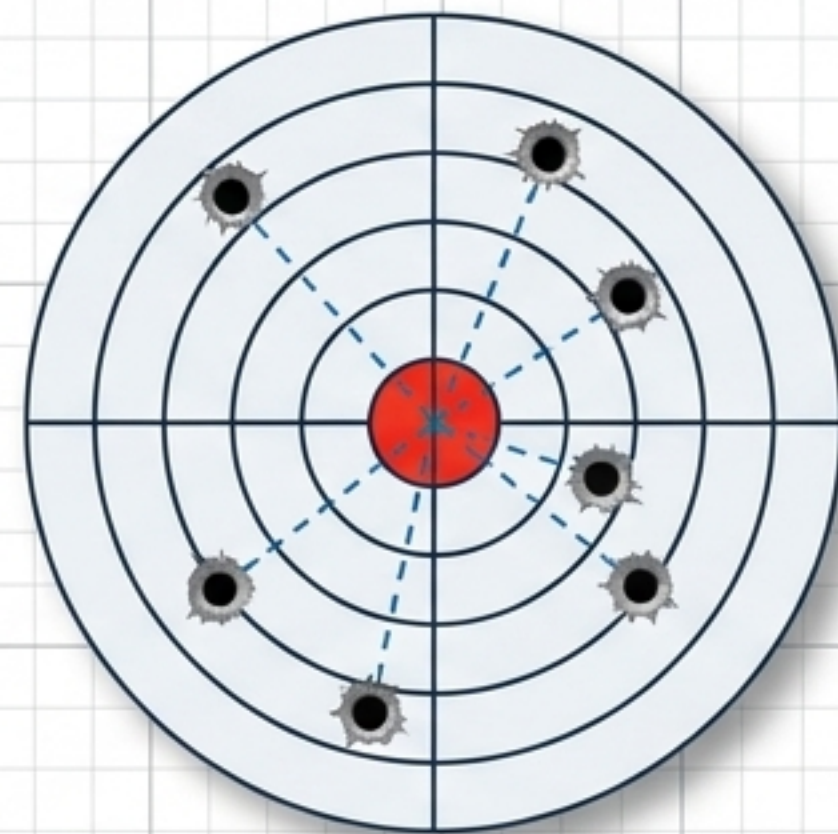


當同一化工系統面對不同生產目標 (多個 b) 時，預分解是唯一正解。

過確定系統：尋找最佳妥協 (lstsq)



化工應用場景：參數迴歸估計、含雜訊的感測器數據融合 (例：5 感測器流率估計)

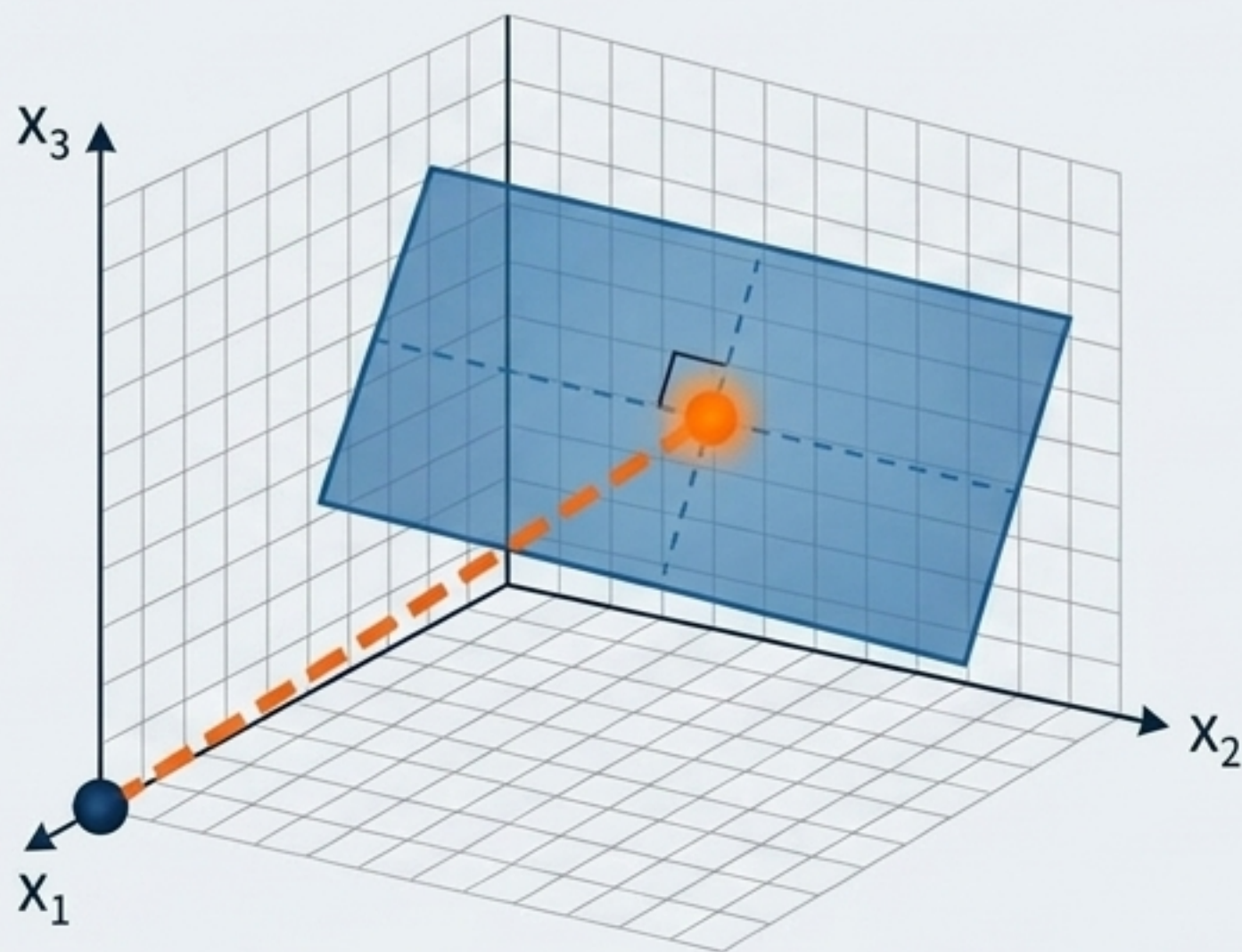


數學條件： $m > n$ (方程式數量 > 未知數，資源多於變數，互相矛盾)

優化目標： $\min \|Ax - b\|_2$ (使殘差的 L2 範數最小)

物理意義：無法完美滿足所有條件，但能找到總距離最短的最佳妥協點。

低確定系統：最小阻力之路 (pinv)



數學條件： $m < n$ (方程式數量 < 未知數，無窮多解)

優化目標： $\min ||\mathbf{x}||_2$ (最小範數解)

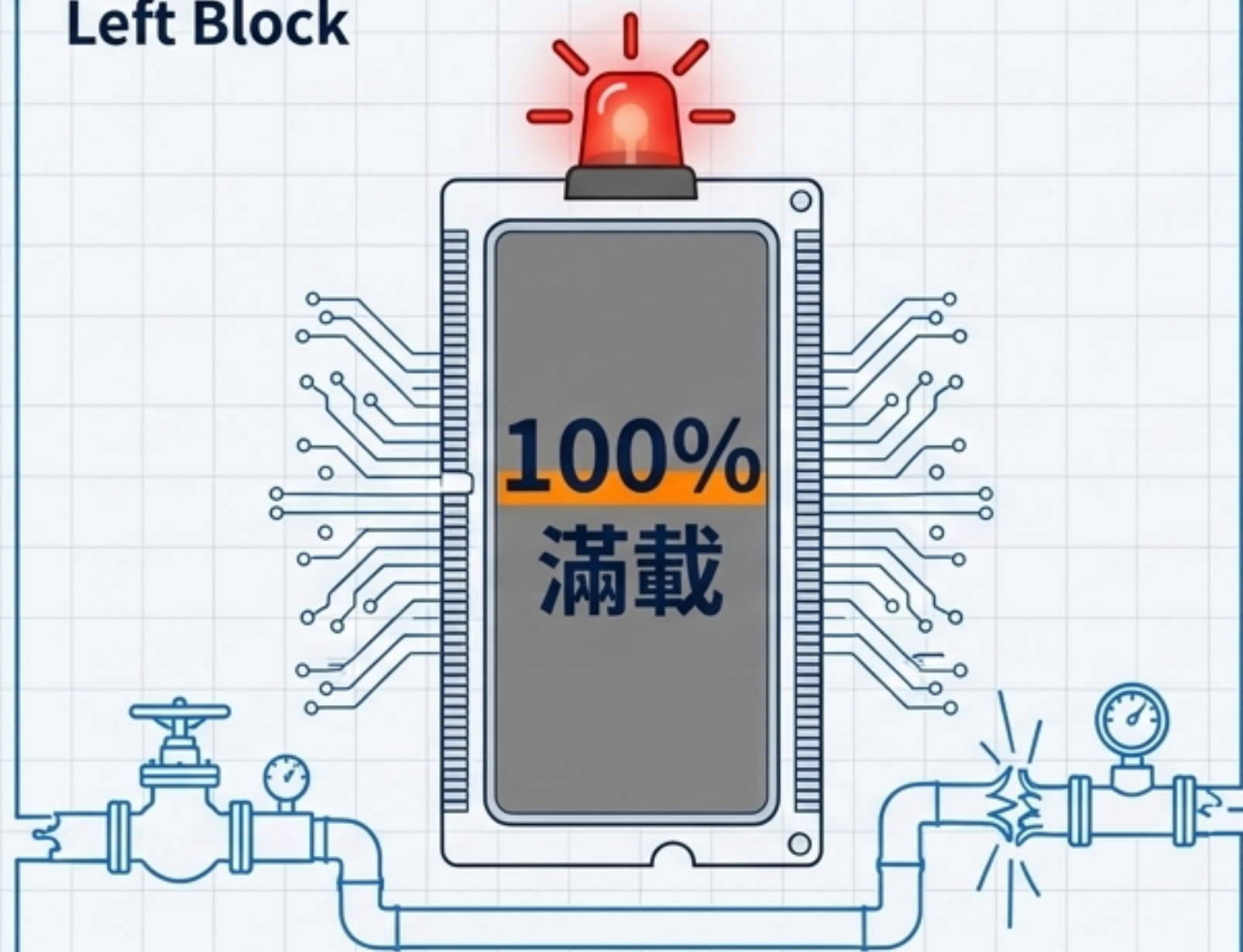
虛擬反矩陣運算： $\mathbf{x} = \mathbf{A}^+ \mathbf{b}$

物理意義：在無數種可能的操作方案中，找到距離原點最近、操作變動量最小的一組解。

化工應用場景：系統具有多餘自由度時（如多條可調回流管線），找出最節能/最穩定的設定值。

微觀世界：為何需要稀疏矩陣？(Sparsity)

Left Block



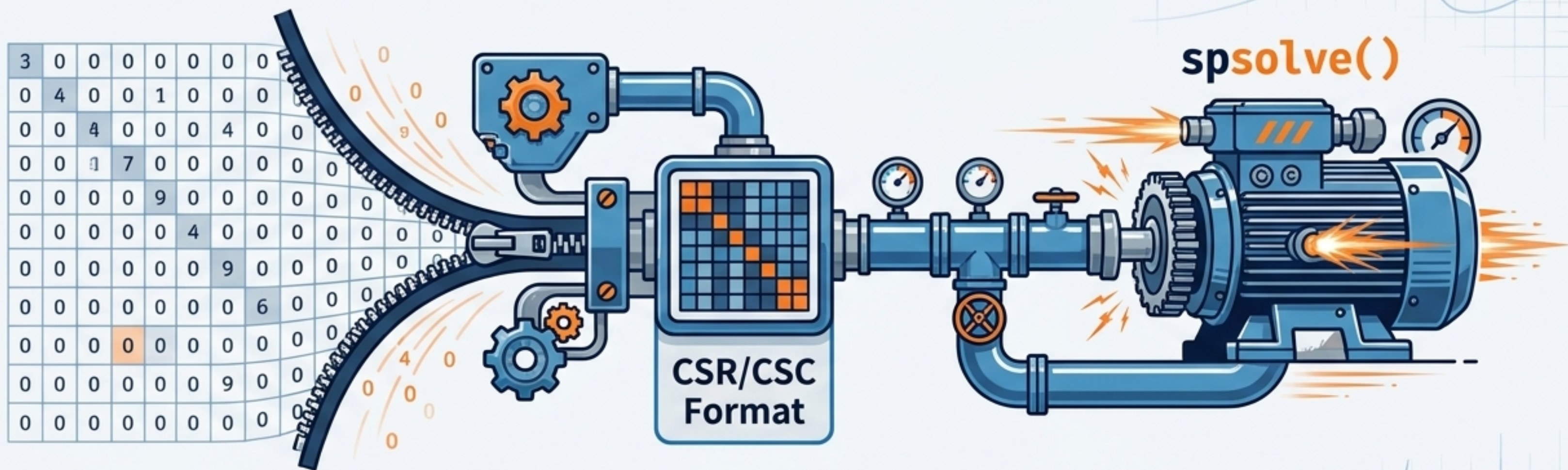
儲存 1,000,000 個元素，面臨算力崩潰

Right Block



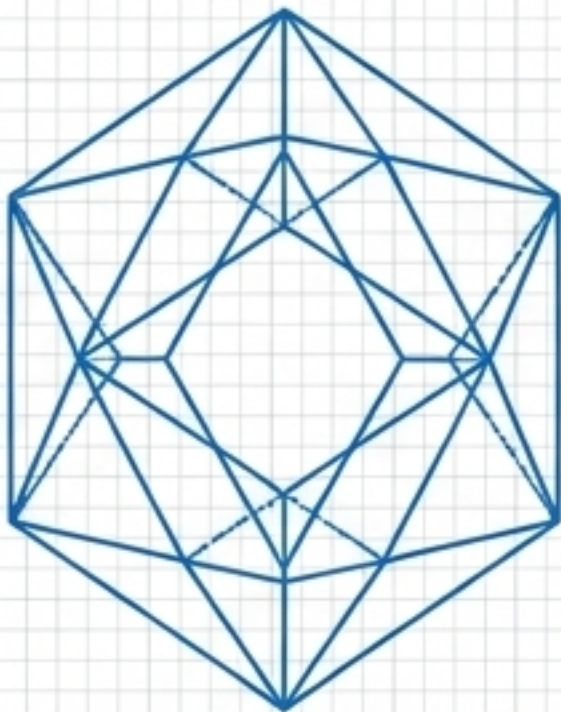
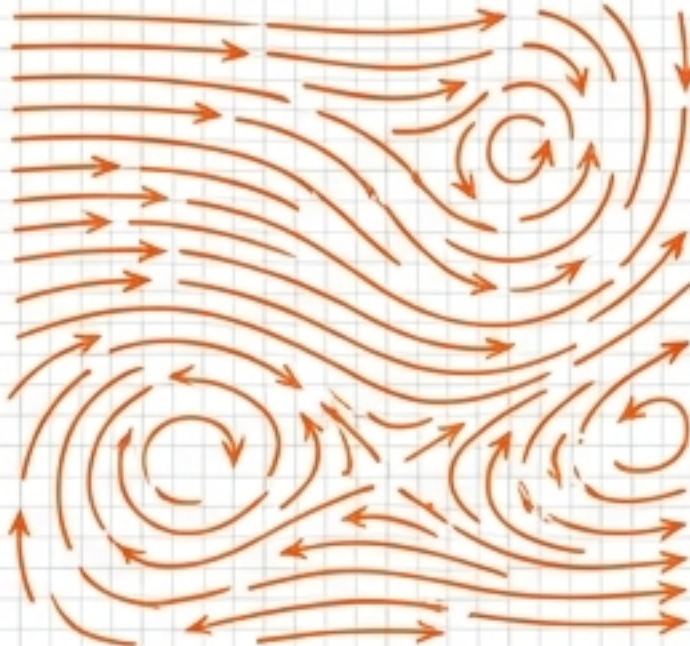
- 案例：一維穩態熱傳導離散化系統 (1000×1000 內部節點)。
- 關鍵數據：非零元素僅 2998 個 -> 矩陣密度 0.30%。
- 結論：使用 **csc** 或 **csc** 壓縮格式儲存，可避開不必要的零運算，大幅節省記憶體並極速提升運算效率。

巨型系統的直接突破：spsolve()



- **定義：** `scipy.sparse.linalg.spsolve()` 是處理巨型稀疏方形矩陣的第一線預設武器。
- **操作提示：** 在輸入引擎前，務必先將矩陣轉為 CSR 或 CSC 格式 (`A.tocsr()`)，以釋放最高運算速度。
- **工程優勢：** 作為直接法，它不需調整超參數，對於化工常見的穩態熱傳導等一維或二維離散化物理系統，能提供極端穩定的極致效率。

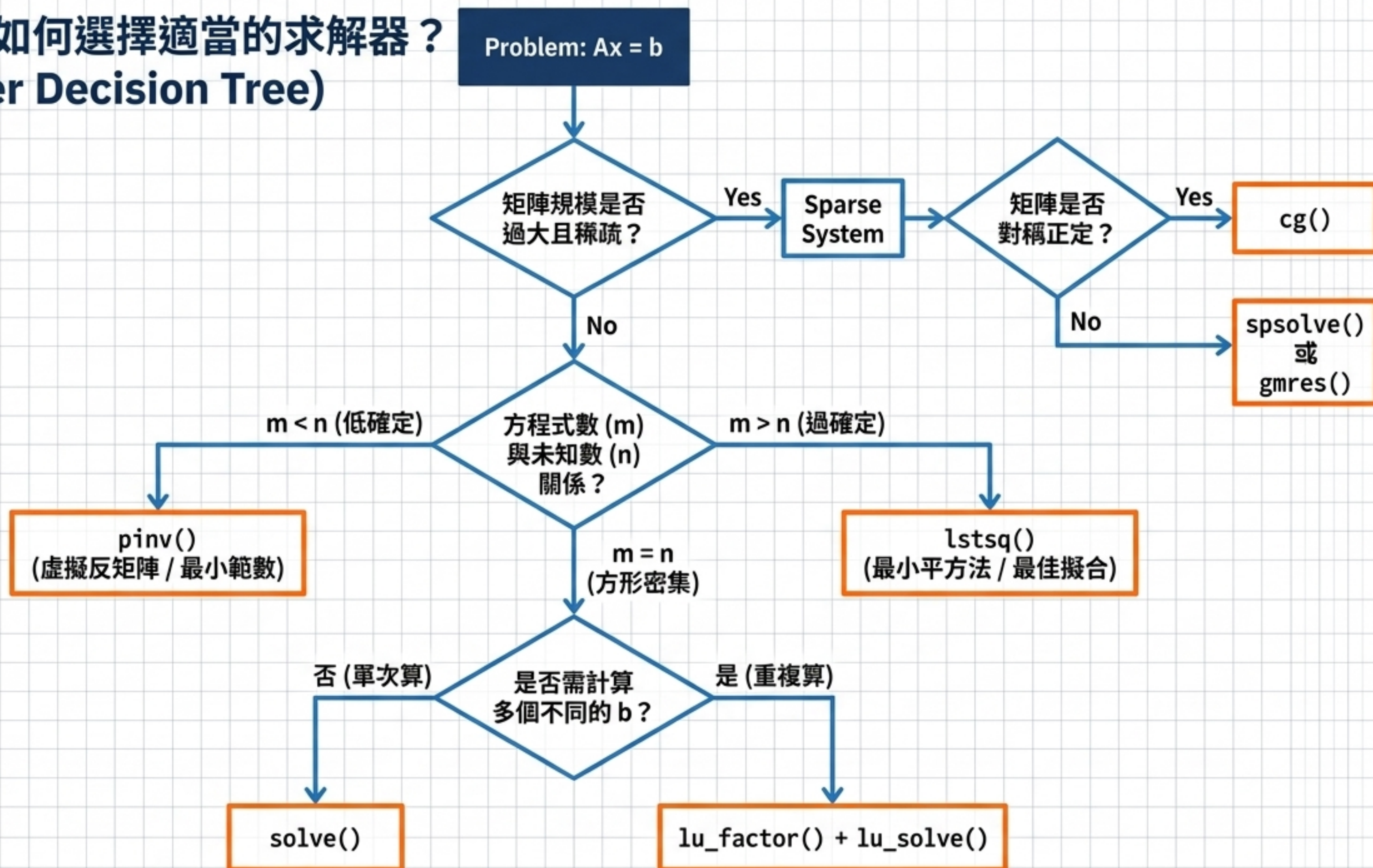
迭代法的雙峰：CG vs. GMRES

CG (共軛梯度法)	GMRES (廣義最小殘差法)
	
- 適用條件：對稱正定矩陣 (SPD)。 - 運算特點：收斂速度極快、記憶體需求極低。	- 適用條件：一般非對稱矩陣。 - 運算特點：通用性最強。記憶體需求隨 restart 參數增加。強烈建議搭配預調節器 (如 ILU) 使用。



極限測試 (5000節點一維擴散)：GMRES 搭配 ILU 預調節僅需 **1.19 ms**，遠較傳統未調節的 CG (417 ms) **快 350 倍**！

終極決策：如何選擇適當的求解器？ (The Master Decision Tree)



實務建議與最佳實踐 (Best Practices)

工程師查檢表 (Checklist)

- ✓ 計算殘差 (Residuals)：永遠檢查 $\|Ax - b\|$ 是否在合理容許範圍內。
- ✓ 物理意義驗證：求解後確認結果是否符合質量/能量守恆，且不可出現負的濃度或絕對溫度。
- ✓ 捕捉例外狀況：使用 `try-except` 區塊捕捉 `LinAlgError`，防止奇異矩陣導致整個系統崩潰。
- ✓ 監控條件數 (Condition Number)：若 $\kappa > 10^{10}$ 為嚴重病態系統，必須重新審視原始物理模型的假設是否合理。

下一步 (Next Step)：
開啟 `Jupyter Notebook`
立即實作 Unit 06 化工物料與能量平衡問題！